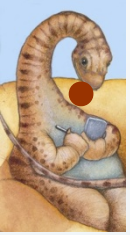
Interprocess Communication

Shatabdi Roy Moon
Lecturer, Dept. of CSE



- **Independent Processes** operating concurrently on a systems are those that can neither affect other processes or be affected by other processes.
- **Cooperating Processes** are those that can affect or be affected by other processes. There are several reasons why cooperating processes are allowed:
  - **Information Sharing-** There may be several processes which need access to the same file for example. (e.g. pipelines)
  - **Computation speedup-** Often a solution to a problem can be solved faster if the problem can be broken down into subtasks to be solved simultaneously (particularly when multiple processors are involved)
  - **Modularity-** The most efficient architecture may be to break a system down into cooperating modules. (E.g. databases with a client server architecture)
  - **Convenience-** Even a single user may be multitasking, such as editing, compiling, printing, and running the same code in different windows.

- **Cooperating processes** require some type of interprocess communication, which is most commonly one of **two types**: **Shared Memory** systems or **Message Passing** systems. Figure 3.12 illustrates the difference between the two systems:

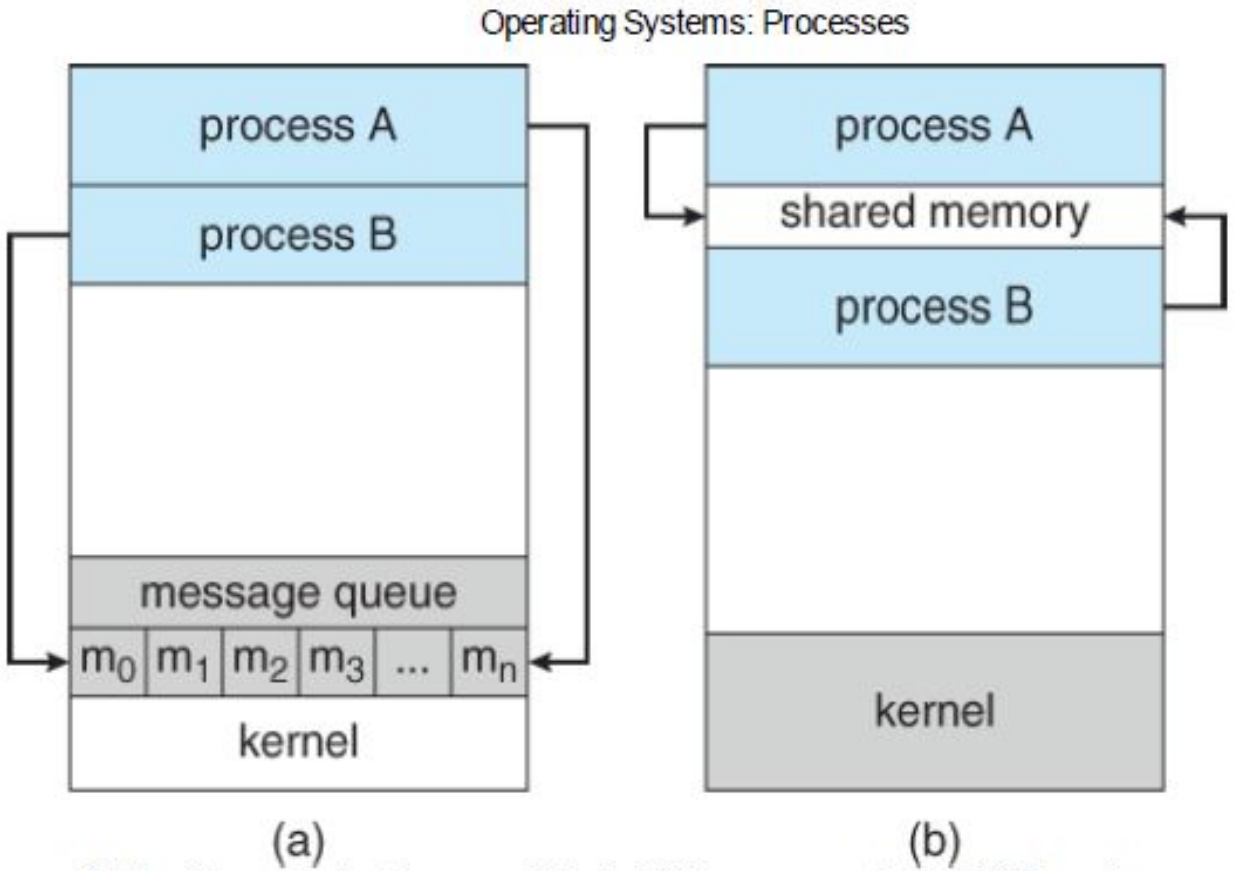


Figure 3.12 - Communications models: (a) Message passing. (b) Shared memory.

- **Shared Memory** is faster once it is set up, because no system calls are required and access occurs at normal memory speeds. However it is more complicated to set up, and doesn't work as well across multiple computers. Shared memory is generally preferable when large amounts of information must be shared quickly on the same computer.

- **Message Passing** requires system calls for every message transfer, and is therefore slower, but it is simpler to set up and works well across multiple computers. Message passing is generally preferable when the amount and/or frequency of data transfers is small, or when multiple computers are involved.

Shared Memory Systems

- In general the memory to be shared in a shared memory system is initially within the address space of a particular process, which needs to make system calls in order to make that memory publicly available to one or more other processes.
- Other processes which wish to use the shared memory must then make their own system calls to attach the shared memory area onto their address space.
- Generally a few messages must be passed back and forth between the cooperating processes first in order to set up and coordinate the shared memory access.

Producer Consumer Example Using Shared Memory

- This is a classic example, in which one process is producing data and another process is consuming the data. (In this example in the order in which it is produced, although that could vary.)
- The data is passed via an intermediary buffer, which may be either **unbounded** or **bounded**. With a bounded buffer the producer may have to wait until there is space available in the buffer, but with an unbounded buffer the producer will never need to wait.
- The consumer may need to wait in either case until there is data available.

Message Passing Systems

- Message passing systems must support at a minimum system calls for "**send message**" and "**receive message**".
- A communication link must be established between the cooperating processes before messages can be sent.
- There are three key issues to be resolved in message passing systems as further explored in the next three subsections:
  - **Direct or indirect communication (naming)**
  - **Synchronous or asynchronous communication**
  - **Automatic or explicit buffering**.

Naming

Direct communication :

- **send(P, message)**—Send a message to process P.
- **receive(Q, message)**—Receive a message from process Q.

- The sender must know the name of the receiver to which it wishes to send a message.
- There is a one-to-one link between every sender-receiver pair.
- For **Symmetric** communication, the receiver must also know the specific name of the sender from which it wishes to receive messages. For **Asymmetric** communications, this is not necessary. Ex:
  - **send(P, message)**—Send a message to process P.
  - **receive(id, message)**—Receive a message from any process. The variable id is set to the name of the process with which communication has taken place.

Naming

Indirect communication :

- **send(A, message)**—Send a message to mailbox A.
- **receive(A, message)**—Receive a message from mailbox A.

- Indirect communication uses shared mailboxes, or ports.
- Multiple processes can share the same mailbox or boxes.
- Only one process can read any given message in a mailbox. Initially the process that creates the mailbox is the owner, and is the only one allowed to read mail in the mailbox, although this privilege may be transferred.

  (Of course the process that reads the message can immediately turn around and place an identical message back in the box for someone else to read, but that may put it at the back end of a queue of messages)
- The OS must provide system calls to create and delete mailboxes, and to send and receive messages to/from mailboxes.

Synchronization

- Either the sending or receiving of messages (or neither or both) may be either **blocking** or **non-blocking**.
  - **Blocking send**. The sending process is blocked until the message is received by the receiving process or by the mailbox.
  - **Non-blocking send**. The sending process sends the message and resumes operation.
  - **Blocking receive**. The receiver blocks until a message is available.
  - **Non-blocking receive**. The receiver retrieves either a valid message or a null.

Buffering

Messages are passed via queues, which may have one of **three** capacity configurations:

1. **Zero capacity -** Messages cannot be stored in the queue, so senders must block until receivers accept the messages.
2. **Bounded capacity -** There is a certain predetermined finite capacity in the queue. Senders must block if the queue is full, until space becomes available in the queue, but may be either blocking or non-blocking otherwise.
3. **Unbounded capacity -** The queue has a theoretical infinite capacity, so senders are never forced to block.